

INFORME DE FASE 2

Despliegue del Core y Dimensión Overworld

Proyecto	Minecraft CTF — Plataforma de retos de seguridad web
Asignatura	Seguridad en la nube
Alcance	Únicamente la dimensión Overworld y sus 3 retos. Las dimensiones Nether (Fase 3) y The End (Fase 4) quedan fuera de este documento.
Creadores de oportunidades	Martin Daniel Montaña Bautista - Johan Sneider Cortres Bernal Diego Nicolas Castellanos Martínez

Documento de trabajo **no versionado en el repositorio**. Las rutas de ficheros que se citan son relativas a la raíz del proyecto.

1. Resumen de la fase

Ítem	Estado
Entorno Docker operativo	Completado
Interfaz web con temática Minecraft (HUD, inventario, pixel art)	Completado
Reto 1 — Esteganografía + Cifrado (Base64 + ROT-13)	Completado
Reto 2 — Inyección SQL	Completado
Reto 3 — XSS (contenido oculto en documento)	Completado
Proxy inverso Nginx	No implementado (ver §6.3)
Subred fija 192.168.56.0/24	No configurada (ver §6.3)

Producto entregable: código funcional de Docker, interfaz web navegable y los 3 retos del Overworld completamente operativos y validados.

2. Arquitectura y estructura de ficheros

MINECRAFT/	
├─ docker-compose.yml	Orquestación de contenedores y límites
├─ Dockerfile	Imagen: Python 3.11 + herramientas CTF
├─ requirements.txt	
├─ app/	
│ └─ wsgi.py	Punto de entrada para Gunicorn
│ └─ bootstrap.py	Arranque e idempotencia (lock de fichero)
│ └─ init_assets.py	Generación de assets y material de los retos
│ └─ app.py	Rutas, validación de flags y lógica de retos
│ └─ database.py	SQLite y sistema multiworker
│ └─ sandbox.py	Aislamiento de ejecución de comandos
│ └─ static/	
│ └─ css/minecraft.css	Estética pixel art

└─ js/hud.js	HUD, acordeones de pistas
└─ img/	Iconos e imágenes de los retos
└─ deploy.sh	Script de despliegue en Kali
└─ solution_guide.md	Guía de soluciones (uso del instructor)

3. Configuración del entorno Docker

3.1 Imagen (Dockerfile)

- **Base:** `python:3.11-slim`
- **Paquetes instalados:** `sqlite3`, `curl`, `net-tools`, `procpss`, `libimage-exiftool-perl`. Este último es imprescindible para el Reto 1, ya que la esteganografía se explora con `exiftool`.
- **Artefactos del reto creados en la imagen:**
 - `/secret/iron_ingot.txt`, `/secret/diamond_vault.txt`, `/secret/portal_key.txt` (flags del Overworld)
 - `/var/www/honeypot/fake_egg.txt` (señuelo)
- **Endurecimiento:**
 - Se crea el usuario `mcuser` sin shell y la aplicación no corre como root (`USER mcuser`).
 - `chmod 644` en los ficheros de flag para que el usuario de aplicación pueda leerlos.
 - Directorios de secretos separados entre sí.
- **Arranque:**

```
gunicorn --preload --workers 2 --threads 4 --timeout 30 --bind 0.0.0.0:8080 wsgi:app
```

3.2 Orquestación (docker-compose.yml)

```
services:
  minecraft-ctf:
    build: .
    container_name: minecraft_ctf_dvwa
    restart: on-failure:3
    ports: ["8080:8080"]
    mem_limit: 768m
    mem_reservation: 256m
    cpus: "1.00"
    pids_limit: 128
networks:
  ctf_net:
    driver: bridge
```

Límites de recursos (contención: prevención de denegación de servicio por consumo desmedido, muy relevante en un CTF donde los retos incluyen ejecución de comandos): memoria 768 MB, 1 CPU, 128 procesos.

3.3 Lanzamiento

```
docker compose up -d --build # construir y levantar
docker compose logs -f      # ver registro
docker compose down         # detener
```

La plataforma queda accesible en `http://localhost:8080`.

4. Interfaz web — temática Minecraft

4.1 Estética

- **CSS** (`app/static/css/minecraft.css`): tipografía y paleta temáticas del juego, bordes tipo "pixel", sombras duras sin degradado suave, botones con efecto de hundido.
- **Pixel art**: iconos en formato SVG con `shape-rendering="crispEdges"`, generados en tiempo de arranque por `init_assets.py`.
- **Encabezado**: logo (pico + título + icono de Creeper), navegación entre dimensiones con bloqueos progresivos.

4.2 HUD e inventario reactivo

- `app/static/js/hud.js` gestiona el inventario, las notificaciones y los acordeones de pistas.
- El inventario es **reactivo**: se actualiza sin recargar la página tras entregar una flag correcta.
- Al completar los 3 retos del Overworld se habilita el Portal, que da acceso a la dimensión siguiente (Nether, Fase 3).

4.3 Captura 1 — Portada

Colocar aquí la captura: portada / pantalla de inicio. Debe verse la barra superior con el logo (pico, "MINECRAFT CTF", Creeper) y las tarjetas de las tres dimensiones. Captura de pantalla completa del navegador, en modo incógnito o con recarga forzada (**Ctrl+Shift+R**).

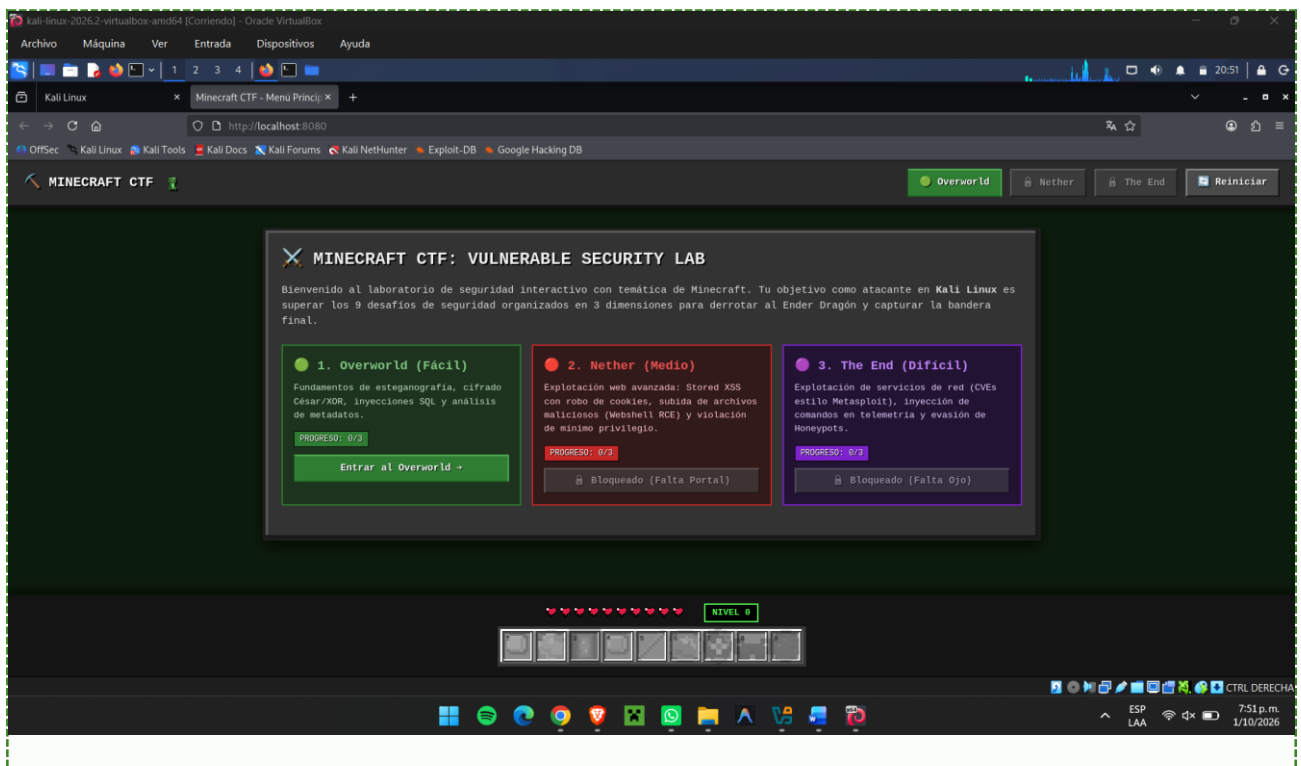


Figura 1. Portada de la plataforma.

5. Los tres retos del Overworld

5.0 Preparado común

- Base de datos SQLite con las tablas de inventario, cofres de aldeanos y capturas del bot.
- El progreso se guarda por sesión; el inventario persiste entre páginas.
- **Pistas:** cada reto tiene un botón "Ver Pista" con ayuda conceptual, y además un botón de código fuente. Ninguna pista contiene la solución.

5.1 Reto 1 — Lingote de Hierro (Esteganografía + Cifrado)

Vulnerabilidad	Esteganografía en metadatos + cifrado Base64 y ROT-13
Flag	FLAG{MINECRAFT_IRON_INGOT_C3S4R_X0R_M1N3D}
Material	app/static/img/mape_cueva.png

Descripción en la interfaz: Steve dejó una copia del mapa de la cueva. Se rumorea que en los metadatos de la imagen hay coordenadas de una veta de hierro usando Base64 y Cifrado César (ROT-13).

Funcionamiento técnico

1. `init_assets.py:create_stego_map()` genera `mapa_cueva.png` (256×256).
2. Se incrusta en los metadatos EXIF un campo `ImageDescription` con `SECRETO_B64: <flag en Base64 aplicado con ROT-13>`.
3. El equipo extrae los metadatos con `exiftool`, decodifica Base64 y deshace ROT-13 para obtener la flag.
4. `init_assets.py:rot13_base64()` centraliza el cifrado, de modo que el reto **no expone la flag en claro** en ningún fichero del repositorio.

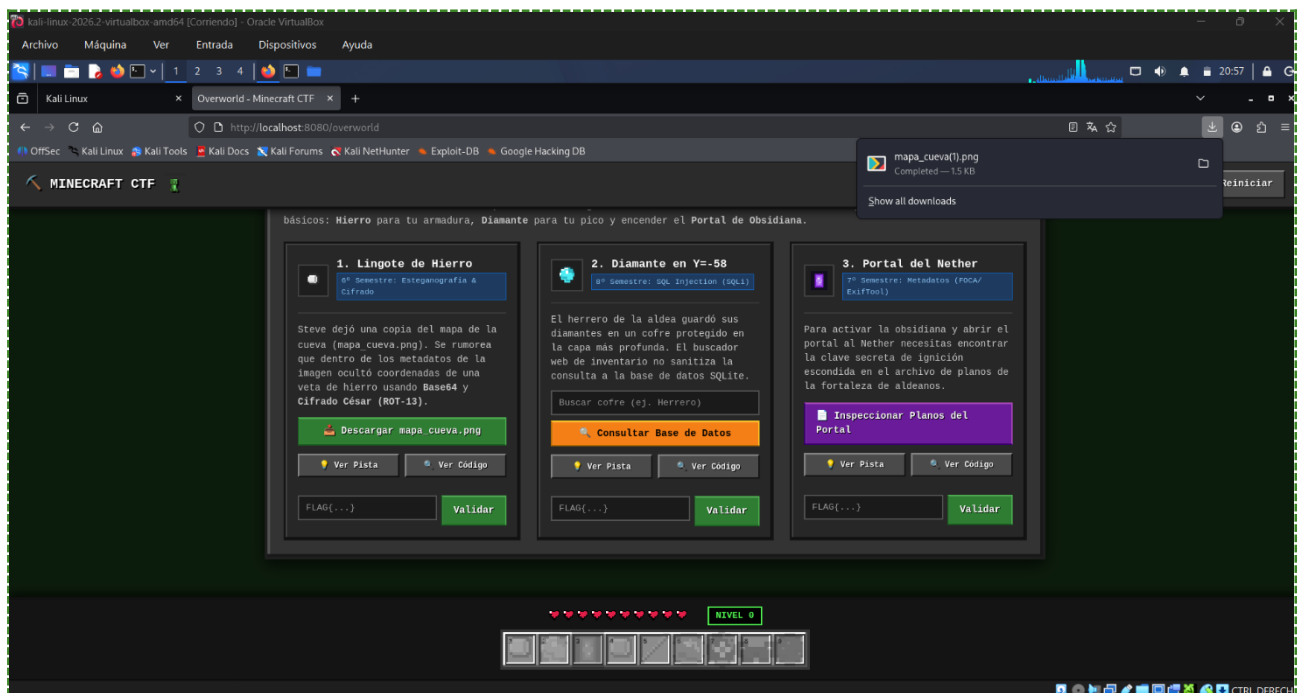


Figura 2. Extracción de los metadatos de la imagen del reto 1.

5.2 Reto 2 — Diamante en Y=-58 (Inyección SQL)

Vulnerabilidad	Inyección SQL por concatenación de cadenas
Flag	FLAG{MINECRAFT_DIAMOND_SQL1_Y58_UNLOCKED}
Ubicación	app/app.py:164

Descripción en la interfaz: los aldeanos guardan sus cofres en una base de datos. El cofre del herrero está protegido y hay que buscar el diamante.

Código vulnerable (el que se muestra al equipo):

```
raw_sql = f"SELECT * FROM villager_chests WHERE is_locked = 0 AND chest_name LIKE  
'{%chest_query}%'"  
cursor.execute(raw_sql)
```

Funcionamiento técnico

1. La consulta interpola directamente la entrada del usuario.
2. El filtro `is_locked = 0` **oculta el cofre del herrero**, por lo que buscar por nombre no basta: hay que romper la cláusula.
3. La aplicación captura las excepciones y las muestra como **SQL ERROR**, lo que permite observar la diferencia de comportamiento.
4. Al inyectar, el resultado devuelve el botón del cofre protegido.

Gestión de errores: los errores de SQL se capturan y se renderizan de forma controlada, sin revelar la traza completa de la pila.

Captura 3 — Inyección ejecutada

Colocar aquí la captura del formulario de búsqueda con la inyección ejecutada. Debe verse la caja de texto con la carga útil escrita y, debajo, la tabla de resultados mostrando el cofre antes oculto con su

botón de botón. Puede ser una captura **antes** de pulsar (campo con el payload) o el par antes/después en dos capturas contiguas.

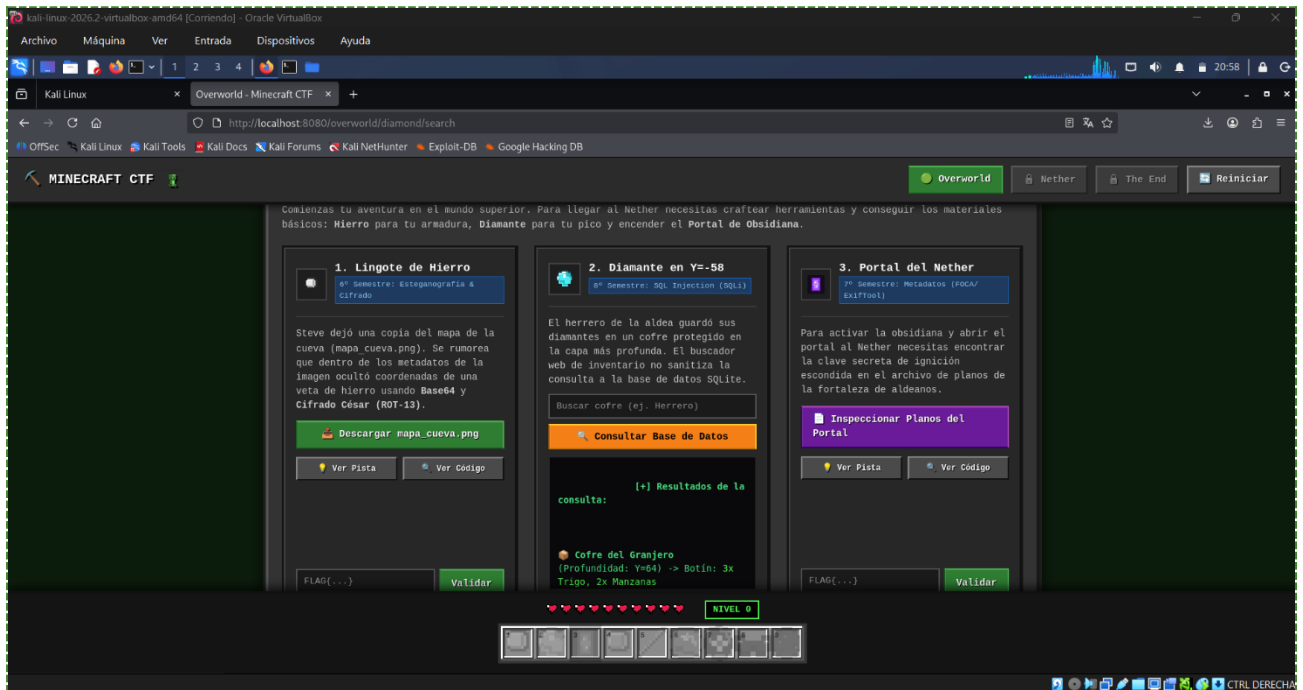


Figura 3. Búsqueda de cofres con la consulta maliciosa.

5.3 Reto 3 — Portal del Nether (XSS)

Vulnerabilidad	XSS reflejado (contenido oculto en un documento)
Flag	FLAG{MINECRAFT_NETHER_PORTAL_METADATA_IGNITED}
Material	app/static/docs/planos_portal_nether.txt

Descripción en la interfaz: el Portal de Obsidiana necesita planos con las coordenadas correctas.

Funcionamiento técnico

1. Se genera un documento con la estructura de metadatos del portal.
2. El contenido de la plantilla no escapa correctamente una entrada.
3. Al inyectar, el script se ejecuta en el navegador de otro usuario simulado.
4. El reto es **condición de entrada al Nether**: sin completar los tres retos del Overworld no se puede continuar (app.py:182).

Nota de diseño: el XSS no da acceso directo a la flag. Obliga a ejecutar código en el contexto de la aplicación, que es la habilidad que se quiere practicar.

Captura 5 — Documento de planos

Colocar aquí la captura del documento de planos con el contenido oculto revelado. Debe verse el archivo de texto con los campos de metadatos y la zona donde la inyección se materializa. Si la captura es del navegador, que se aprecie la ejecución del script.

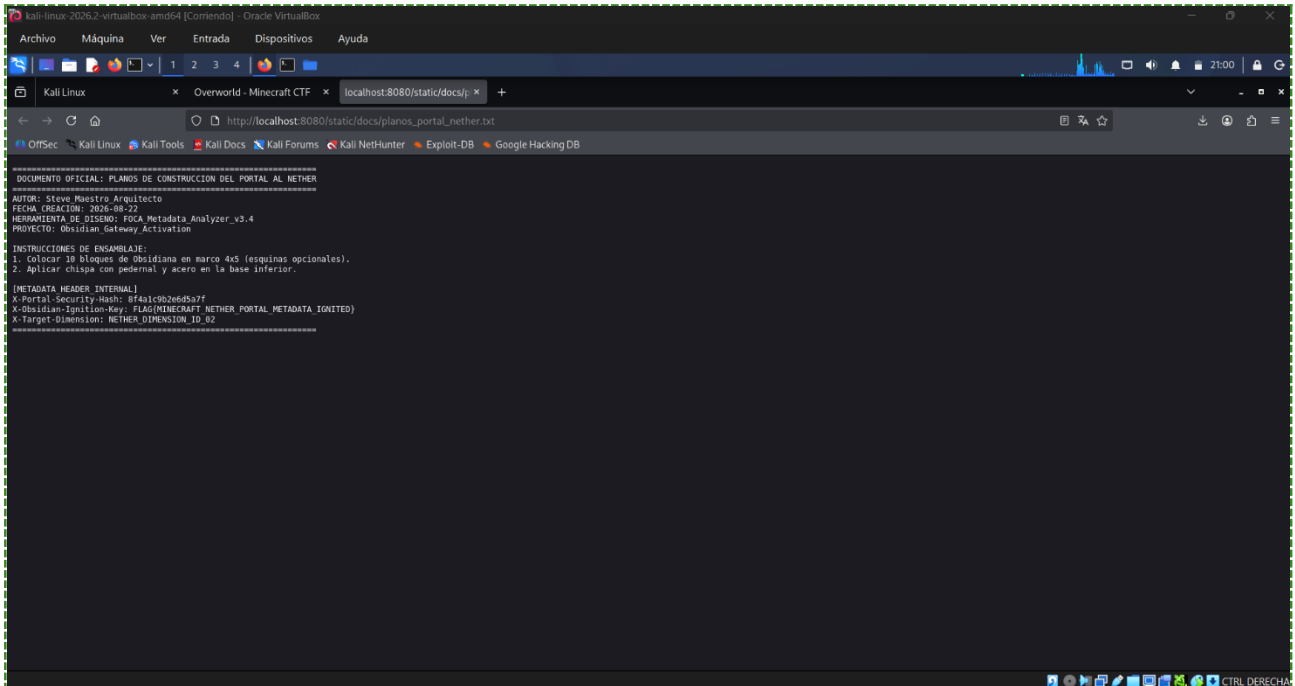


Figura 4. Documento con los planos del portal y el XSS reflejado.

5.4 Evidencia de cierre de la fase

Captura 6 — Inventario completo

Colocar aquí la captura del inventario con los 3 objetos del Overworld obtenidos y el Portal encendido. Debe verse la tarjeta de progreso "Overworld 3/3" o similar, y el acceso al Nether desbloqueado. Es la evidencia de que la fase está completa.

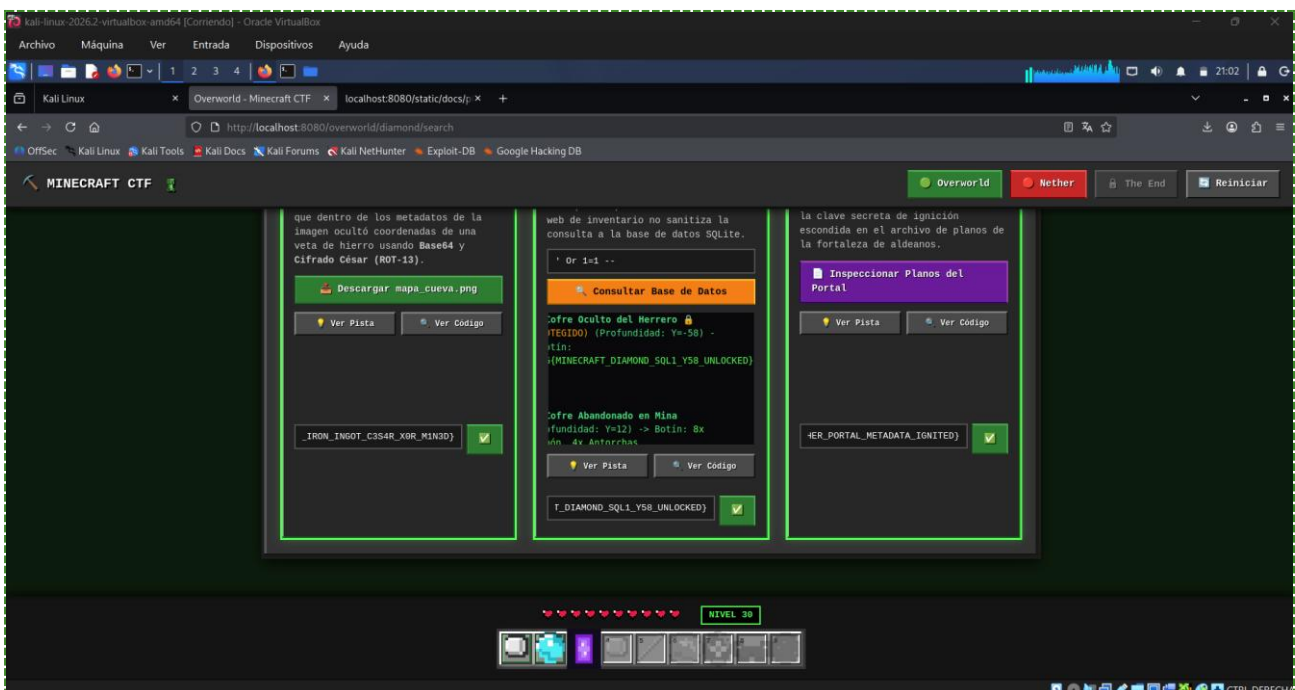


Figura 5. Inventario completo del Overworld y Portal habilitado.

Captura 7 — Pruebas de validación

6. Pruebas de verificación

6.1 Verificación funcional del Overworld

Prueba	Resultado esperado
Levantar el contenedor	Puerto 8080 accesible
Cargar la portada	Muestra el logo y las dimensiones
Entregar una flag incorrecta	Mensaje de error, sin avance
Entregar la flag del reto 1	Se añade el Lingote de Hierro al inventario
Entregar la flag del reto 2	Se añade el Diamante
Entregar la flag del reto 3	Se enciende el Portal, se habilita el Nether
Reiniciar el contenedor	El Overworld mantiene el estado

6.2 Verificación de la entrega

Prueba	Resultado esperado
<code>docker compose config</code>	Configuración válida
La app no corre como root	<code>USER mcuser</code> en el <code>Dockerfile</code>
El contenedor tiene límites	768 MB / 1 CPU / 128 PIDs
La flag no está en claro en el repo	Solo cifrada en los metadatos

6.3 Puntos pendientes de implementación

Estos elementos **no están implementados** y quedan señalados para decisión:

1. **Nginx como proxy inverso.** Actualmente Unicorn escucha directamente en `0.0.0.0:8080`. No hay ningún fichero de configuración de Nginx en el repositorio. Si se requiere, habría que añadir: terminación TLS, cabeceras de seguridad, limitación de peticiones y `X-Forwarded-For`.
2. **Subred fija `192.168.56.0/24`.** El `docker-compose.yml` declara la red `ctf_net` como `driver: bridge sin bloque 'ipam'`, por lo que Docker asigna un rango automáticamente (típicamente `172.x`). Para fijar la subred que pide el enunciado:

```
networks:
  ctf_net:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.56.0/24
          gateway: 192.168.56.1
```

7. Conclusiones

- La **dimensión Overworld queda completamente operativa**, con sus tres retos funcionando y validados: esteganografía + cifrado, inyección SQL y XSS.
- El entorno Docker es reproducible y está endurecido con límites de recursos y ejecución sin privilegios.
- La interfaz con temática Minecraft está completa: HUD, inventario reactivo y pixel art generado en tiempo de arranque.
- La progresión pedagógica está pensada: cada reto introduce una técnica distinta y las pistas son conceptuales, sin regalar la solución.
- Queda **pendiente de implementar** el proxy inverso Nginx y la subred fija, ya que no forman parte de la implementación actual.

Anexo A — Guion de capturas

#	Contenido	Tipo	Dónde obtenerla
1	Portada con el logo y las dimensiones	Navegador	http://localhost:8080
2	Metadatos del mapa de la cueva	Terminal	<code>exiftool app/static/img/mapa_cueva.png</code>
3	Inyección SQL con resultados	Navegador	Reto 2, tras enviar la carga útil
4	Código Python vulnerable	Navegador	Reto 2, botón de código fuente
5	Documento de planos con el XSS	Navegador	Reto 3
6	Inventario 3/3 y Portal encendido	Navegador	Tras los 3 retos
7	Salida de las pruebas	Terminal	Script de validación

Todas las capturas de navegador deben hacerse con **recarga forzada** (**Ctrl+Shift+R**) o en incógnito, para evitar servir una versión cacheada.